

```

%%writefile cuda_program.cu
#include <stdio.h>
#include <cuda.h>
#include <time.h>

#define N 1024
#define SIZE 32    // for matrix (32x32)

// ----- VECTOR ADDITION -----

// CPU version
void vectorAddCPU(int *a, int *b, int *c)
{
    for(int i = 0; i < N; i++)
        c[i] = a[i] + b[i];
}

// GPU kernel
__global__ void vectorAddGPU(int *a, int *b, int *c)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i < N)
        c[i] = a[i] + b[i];
}

// ----- MATRIX MULTIPLICATION -----

// CPU version
void matrixMulCPU(int A[SIZE][SIZE], int B[SIZE][SIZE], int C[SIZE][SIZE])
{
    for(int i = 0; i < SIZE; i++)
        for(int j = 0; j < SIZE; j++)
        {
            C[i][j] = 0;
            for(int k = 0; k < SIZE; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
}

// GPU kernel
__global__ void matrixMulGPU(int *A, int *B, int *C)
{
    int row = threadIdx.y;
    int col = threadIdx.x;

    int sum = 0;

    for(int k = 0; k < SIZE; k++)
        sum += A[row * SIZE + k] * B[k * SIZE + col];

    C[row * SIZE + col] = sum;
}

// ----- MAIN FUNCTION -----

int main()
{
    // ----- VECTOR ADDITION -----
    int a[N], b[N], c_cpu[N], c_gpu[N];
    int *d_a, *d_b, *d_c;

    for(int i = 0; i < N; i++)
    {
        a[i] = rand() % 100;
        b[i] = rand() % 100;
    }

    // CPU timing
    clock_t start_cpu = clock();
    vectorAddCPU(a, b, c_cpu);
    clock_t end_cpu = clock();

    // GPU memory
    cudaMalloc(&d_a, N*sizeof(int));
    cudaMalloc(&d_b, N*sizeof(int));
    cudaMalloc(&d_c, N*sizeof(int));

    cudaMemcpy(d_a, a, N*sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(int), cudaMemcpyHostToDevice);

    // GPU timing

```

```

// CPU timing
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);

vectorAddGPU<<<(N+255)/256, 256>>>(d_a, d_b, d_c);

cudaEventRecord(stop);
cudaEventSynchronize(stop);

float gpu_time;
cudaEventElapsedTime(&gpu_time, start, stop);

cudaMemcpy(c_gpu, d_c, N*sizeof(int), cudaMemcpyDeviceToHost);

double cpu_time = ((double)(end_cpu - start_cpu)) / CLOCKS_PER_SEC;

printf("Vector Addition:\n");
printf("CPU Time = %f sec\n", cpu_time);
printf("GPU Time = %f ms\n", gpu_time);
printf("Speedup = %f\n\n", cpu_time / (gpu_time / 1000.0));

// ----- MATRIX MULTIPLICATION -----

int A[SIZE][SIZE], B[SIZE][SIZE], C_cpu[SIZE][SIZE], C_gpu[SIZE][SIZE];
int *d_A, *d_B, *d_C;

for(int i = 0; i < SIZE; i++)
    for(int j = 0; j < SIZE; j++)
    {
        A[i][j] = rand() % 10;
        B[i][j] = rand() % 10;
    }

// CPU timing
start_cpu = clock();
matrixMulCPU(A, B, C_cpu);
end_cpu = clock();

cudaMalloc(&d_A, SIZE*SIZE*sizeof(int));
cudaMalloc(&d_B, SIZE*SIZE*sizeof(int));
cudaMalloc(&d_C, SIZE*SIZE*sizeof(int));

cudaMemcpy(d_A, A, SIZE*SIZE*sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_B, B, SIZE*SIZE*sizeof(int), cudaMemcpyHostToDevice);

dim3 threads(SIZE, SIZE);

cudaEventRecord(start);

matrixMulGPU<<<1, threads>>>(d_A, d_B, d_C);

cudaEventRecord(stop);
cudaEventSynchronize(stop);

cudaEventElapsedTime(&gpu_time, start, stop);

cudaMemcpy(C_gpu, d_C, SIZE*SIZE*sizeof(int), cudaMemcpyDeviceToHost);

cpu_time = ((double)(end_cpu - start_cpu)) / CLOCKS_PER_SEC;

printf("Matrix Multiplication:\n");
printf("CPU Time = %f sec\n", cpu_time);
printf("GPU Time = %f ms\n", gpu_time);
printf("Speedup = %f\n", cpu_time / (gpu_time / 1000.0));

cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);

return 0;
}

```

Overwriting cuda_program.cu

```
!nvcc cuda_program.cu -o cuda_program
```

[Show hidden output](#)

```
!nvidia-smi
```

Wed May 13 10:37:18 2026

NVIDIA-SMI 580.82.07			Driver Version: 580.82.07			CUDA Version: 13.0		
GPU Name		Persistence-M		Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp	Perf	Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	MIG M.
0	Tesla T4		Off	00000000:00:04.0	Off			0
N/A	38C	P8	10W / 70W	0MiB / 15360MiB		0%	Default	N/A
Processes:								
GPU	GI	CI	PID	Type	Process name	GPU Memory		
	ID	ID				Usage		
No running processes found								

!./cuda_program

Vector Addition:

CPU Time = 0.000005 sec

GPU Time = 102.891518 ms

Speedup = 0.000049

Matrix Multiplication:

CPU Time = 0.000121 sec

GPU Time = 0.033504 ms

Speedup = 3.611509